

TimblServer: Tilburg Memory-Based Learner Server

version 1.20

Manual

Ko van der Sloot
Centre for Language Studies
Radboud University Nijmegen

URL: <https://language-machines.github.io/timbl>¹

July 3, 2026

¹This document is available from https://github.com/LanguageMachines/timblserver/blob/master/docs/TimblServer_1.20_Manual.pdf. All rights reserved Induction of Linguistic Knowledge, Tilburg University and CLiPS, University of Antwerp.

Contents

1	GNU General Public License	1
2	Installation	2
3	Changes	4
3.1	From version 1.1 to 1.20	4
3.2	version 1.1	4
4	Quick-start Tutorial	6
5	Software usage and options	7
5.1	Command line options	7
5.1.1	Server options	7
5.1.2	Configuration file	8
6	Server protocols	9
6.1	The TCP protocol	9
6.2	The HTTP protocol	10
6.3	The JSON protocol	11
7	The timblclient program	13

Preface

This reference guide describes the server interface to TiMBL, the Tilburg Memory-Based Learner software package. To be able to use TimblServer, a working installation of TiMBL (version 7.0 or later) should be present on your system. For information on the installation and use of TiMBL, see the TiMBL Reference Guide [1] and the TiMBL webpage¹.

TimblServer wraps one or more TiMBL classifiers (*instance bases*) in a network server, so that trained classifiers can be kept in memory and be queried from multiple simultaneous client sessions, locally or over the network. Since version 1.13 the server speaks three protocols: a line-based TCP protocol, an HTTP protocol with XML output, and a JSON protocol.

Although this software has been tested for a long time in our research groups, it may still contain bugs and inconsistencies in some places. We would appreciate it if you would file bug reports, ideas about enhancements of the software and the manual, and any other comments you might have, as issues at <https://github.com/LanguageMachines/timblserver/issues>.

This reference guide is structured as follows. In Chapter 1 you can find the terms of the license according to which you are allowed to use TimblServer. The subsequent chapter gives some instructions on how to install the TimblServer package on your computer. Chapter 3 lists the changes that have taken place up to the current version. Next, Chapter 4 offers a quick-start tutorial for readers who want to get to work with TimblServer right away. Chapter 5 describes the command line options and the configuration file, and Chapter 6 documents the three server protocols. Chapter 7 describes the `timblclient` test program.

¹<https://language-machines.github.io/timbl>

Chapter 1

GNU General Public License

TimblServer is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.

TimblServer is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with TimblServer. If not, see <<http://www.gnu.org/licenses/>>.

In publication of research that makes use of TimblServer, a citation should be given of: *“Walter Daelemans, Jakub Zavrel, Ko van der Sloot, and Antal van den Bosch (2026). TiMBL: Tilburg Memory Based Learner, version 7.0, Reference Guide. Available from <https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl-7.0.Manual.pdf>”*.

Chapter 2

Installation

The TimblServer source code is hosted on GitHub:

`https://github.com/LanguageMachines/timblserver`

Stable releases can be downloaded as tarballs from

`https://github.com/LanguageMachines/timblserver/releases`

and the development version can be obtained with

```
> git clone https://github.com/LanguageMachines/timblserver
```

The package contains the complete source code (C++) for the TimblServer program, the license, and documentation. The installation should be relatively straightforward on most UNIX systems.

Before installing TimblServer, please make sure the following dependencies are installed on your system:

- `ticcutils`¹
- `timbl` (version 7.0 or later)²
- `libxml2`
- `pkg-config`

Building requires a C++17 compiler (e.g. GCC 7 or later, or a recent Clang). If in doubt about the installed TIMBL version, check the output of the command `timbl -V`.

To compile and install from a cloned repository (> is the command line prompt):

```
> cd timblserver
> bash bootstrap.sh
> ./configure --prefix=<location.to.install>
> make
> make install
```

¹`https://github.com/LanguageMachines/ticcutils`

²`https://github.com/LanguageMachines/timbl`

When building from a release tarball, the `bootstrap.sh` step can be skipped. If you do not use the `--prefix` option, TimblServer will try to install itself in the directory `/usr/local/`. If you do not have `root` access you can specify a different installation location such as `$HOME/install`.

If the process was completed successfully, you should now have executable files named `timblserver` and `timblclient` in the directory `<location_to_install>/bin`, and a library `libtimblserver` in the directory `<location_to_install>/lib`.

Alternatively, TimblServer is packaged by several Linux distributions, and it is included in our LaMachine software distribution³.

The address for problems with the installation, bug reports, comments and questions is our issue tracker at <https://github.com/LanguageMachines/timblserver/issues>.

³<https://proycon.github.io/LaMachine>

Chapter 3

Changes

3.1 From version 1.1 to 1.20

The changes most relevant to users are summarised below; the `NEWS` and `ChangeLog` files in the source repository give the complete per-release history.

- The executables were renamed from `TimblServer` and `TimblClient` to lowercase `timblserver` and `timblclient`, and the library to `libtimblserver` (version 1.2).
- Next to the original line-based TCP protocol, an HTTP protocol was added, which answers `GET` requests with XML; the `protocol` attribute in the configuration file (marked “reserved for future usage” in the 1.1 manual) selects it. See Section 6.2.
- JSON support was added in version 1.13 (2019). With `protocol=json` the server communicates in newline-delimited JSON objects instead of the line-based TCP protocol. The JSON syntax might still change in future versions, so handle with care. See Section 6.3.
- The generic server infrastructure was moved to the `ticcutils` package (versions 1.6–1.10), on which `TimblServer` now depends, and the source repository moved to GitHub (version 1.10).
- The long options `--version` and `--help` are accepted alongside `-V` and `-h` (version 1.11).
- The dependencies on Boost and libtar were removed; `TimblServer` now depends on `ticcutils`, `timbl`, `libicu` and `libxml2` (versions 1.15–1.17).
- Building `TimblServer` now requires a C++17 compiler (version 1.19).

3.2 version 1.1

This is the initial release of a separate `TimblServer`. Compared with the version included in `Timbl` (up to version 6.2) the following is changed:

- Some bugs are fixed (`-C <n>` didn’t work as expected)

- a `--config` option is added to read a configuration file. This file can be used to specify a whole range of separated and unrelated TiMBL experiments all to run on the same TCP port.

Chapter 4

Quick-start Tutorial

The use of TimblServer subsumes a familiarity with TiMBL. The TimblServer command line interface is the same as that of `timbl` with a few additions, exemplified below.

If you want to run the examples and demos from this manual, you should act as follows. We will use the same examples as TiMBL, so copy the files `dimin.train` and `dimin.test` from the TiMBL distribution¹ to some working location. From the directory you copied the files to, issue the following command:

```
> timblserver -f dimin.train -S 7000
```

This will start a TimblServer on port 7000 (if that port is occupied you might need to try another number).

The server can be used (from the same machine) over Telnet like this:

```
> telnet localhost 7000
```

(or, equivalently, `nc localhost 7000`). The server responds:

```
Welcome to the Timbl server.
```

Try to classify an instance:

```
classify =,=,=,=,=,=,=,=,+,p,e,=,?
```

The server should respond with:

```
CATEGORY {T}
```

Type `exit` to close the session. The server keeps running, and must be stopped by killing the `timblserver` process.

¹In the `demos` directory of the TiMBL source tree.

Chapter 5

Software usage and options

5.1 Command line options

The user interacts with TimblServer through the use of command line arguments. When you have installed TimblServer successfully, and you type `timblserver` at the command line without any further arguments, it will print an overview of the most basic command line options:

```
TimBL Server 1.20 (c) CLTS/ILK/CLIPS 1998 - 2026.
Tilburg Memory Based Learner
Centre for Language and Speech Technology, Radboud University
Induction of Linguistic Knowledge Research Group, Tilburg University
CLiPS Computational Linguistics Group, University of Antwerp
based on timbl 7.0

usage:  timblserver --config=config-file
        timblserver -f data-file {-S socket} {-C num}
or see: timblserver -h
        for more options
```

5.1.1 Server options

`--config=<file>` or `-c <file>`: Start a server with a range of TiMBL experiments as specified in the configfile.

This is the **preferred** way to use TimblServer. See Section 5.1.2 for details about the configfile.

`--protocol=[tcp|http|json]`: select the protocol the server speaks (default `tcp`). See Chapter 6. The protocol can also be set in the configfile.

`--pidfile=<file>`: store the pid of the server in 'file'. A full path should be given, otherwise the file will end up relative to '/'.

`--logfile=<file>`: log server messages to 'file'. A full path should be given, otherwise the file will end up relative to '/'.

`--daemonize=[yes|no]`: run the server as a daemon. The default is *yes*.

`-S <portnumber>`: Starts a TiMBL server listening on the specified port number of the local-host. This option is deprecated and only available to be backward compatible with Timbl 6.2 and earlier versions; the `--config` variant is preferred.

`-C <number>` : limit the number of parallel connections to `<number>`. The default is 10. Deprecated, like `-S`.

All other options are passed on to the embedded TiMBL experiment(s); they are documented in the TiMBL Reference Guide [1] and in `timbl(1)`. The long options `--version` and `--help` are accepted as aliases for `-V` and `-h`.

5.1.2 Configuration file

The configfile contains lines of the general format `attribute=value`. Valid attributes are: `port` to set the TCP port, `maxconn` to set the number of parallel connections, `protocol` to select the `tcp`, `http` or `json` protocol (default `tcp`), and the equivalents of the command line options above: `pidfile`, `logfile`, `daemonize`. The attribute `debug` can be set to get (very) verbose logging.

Besides these attribute–value pairs, the configfile contains any number of lines describing TiMBL experiments to be served. The lines take the form of `basename="timbl training options"`. `basename` is a name you can choose to identify the experiment. The `basename` is needed when you call the server to classify an instance.

"timbl training options" is a string of options which are used to train Timbl. For example:

```
port=7000
maxconn=10
protocol=tcp
[[experiments]]
dimin0="-aIB1 +vdi+db+n -f dimin.train"
dimin1="-aIGTREE +vdi+db +D -f dimin.train"
dimin2="-aTRIBL +vdi+db+n -q3 -f dimin.train"
```

If you start `TimblServer` with this configfile, it will start on port 7000, with at most 10 parallel sessions. The server will train 3 different Timbl instances, identified by the names `dimin0`, `dimin1` and `dimin2`.

The optional section header `[[experiments]]` separates the experiment definitions from the global server settings. For backward compatibility it may be omitted, in which case every attribute–value pair that is not a recognised server setting is interpreted as an experiment definition.

Instead of training from a data file with `-f`, an experiment can also load a previously stored instance base with `-i`, which makes the server start up much faster; optional weights (`-w`), probability arrays (`-u`) and value-difference matrices (`--matrixin`) files are read as well, exactly as in standalone `timbl`.

Chapter 6

Server protocols

When the `TimblServer` is running, it is waiting for input on the specified port number. When a client connects on this port number, the server starts a separate thread, processing any given commands, such as to classify a new example. Which protocol the server speaks on that port is determined by the `protocol` setting: `tcp` (the default, Section 6.1), `http` (Section 6.2), or `json` (Section 6.3). One server instance speaks one protocol.

6.1 The TCP protocol

The TCP protocol is a simple line-based protocol, suitable for use with `telnet` or `nc`, and the protocol spoken by `timblclient` (Chapter 7). After accepting the connection, the server first sends a welcome message to the client:

```
Welcome to the Timbl server.
```

When the server hosts more than one base, this is followed by a line listing the available bases:

```
available bases:  dimin0 dimin1 dimin2
```

After this, the server waits for client-side requests. The client can now issue five types of commands: `base`, `classify`, `set` (options), `query` (status), and `exit`. The type of command is specified by the first string of the request line, which can be abbreviated to any prefix of the command, up to one letter (i.e. `b`, `c`, `s`, `q`, `e`); commands are case-insensitive. The command is followed by whitespace and the remainder of the command as described below. Lines starting with `#` are skipped.

```
base basename
```

tell the server to which base the next command(s) refer. The `basename` must be one of the basenames specified in the configfile. If no configfile was specified, i.e. `TimblServer` was started with the old-school `-S` option, then the `basename` is “default”. It is not necessary to specify this default name when testing (therefore preserving backward compatibility with older `Timbl` versions).

```
classify testcase
```

`testcase` is a pattern of features (must have the same number of features as the training set) followed by a category string. E.g.: `small, long, 1, ??`.

Depending on the current settings of the server, it will either return the answer

```
ERROR { explanation }
```

if something has gone wrong, or the answer

```
CATEGORY {category} DISTRIBUTION { category 1 } DISTANCE { 1.000000 } NEIGHBORS  
ENDNEIGHBORS
```

where the presence of the `DISTRIBUTION`, `DISTANCE`, `MATCH_DEPTH`, `CONFIDENCE` and `NEIGHBORS` parts depends upon the current verbosity settings (`+vdb`, `+vdi`, `+vmd`, `+vcf` and `+vn`, respectively). Note that if the last string on the answer line is `NEIGHBORS`, the server will proceed to return lines of nearest neighbor information until it prints the keyword `ENDNEIGHBORS`.

`set option`

where `option` is specified as a string of commandline options. Only the following commandline options are valid in this context: `k`, `m`, `d`, `B`, `L`, `Q`, `w`, `v`, `x`. The setting of an option in this client does not affect the behavior of the server towards other clients. The server replies either with `OK` or with `ERROR {explanation}`.

`query`

queries the server for a list of current settings. Returns a number of lines with status information, starting with a line that says `STATUS`, and ending with a line that says `ENDSTATUS`. For example:

```
STATUS  
FLENGTH           : 0  
MAXBESTS          : 500  
NULL_VALUE        :  
TREE_ORDER        : G/V  
DECAY              : Z  
INPUTFORMAT       : Column  
SEED               : -1  
DECAYPARAM        : 1.000000  
SAMPLE_WEIGHTS    : -  
IGNORE_SAMPLES    : +  
PROBALISTIC       : -  
VERBOSITY         : F  
EXACT_MATCH       : -  
USE_INVERTED      : -  
GLOBAL_METRIC     : Overlap  
METRICS           :  
NEIGHBORS         : 1  
PROGRESS          : 100000  
TRIBL_OFFSET      : 0  
IB2_OFFSET        : 0  
WEIGHTING         : GRW  
ENDSTATUS
```

`exit`

closes the connection between this client and the server.

6.2 The HTTP protocol

With `protocol=http`, the server answers HTTP `GET` requests, so that a trained TiMBL base can be queried directly from a web browser or with tools such as `curl`. The result is an XML

document.

The path of the request selects the base, and the query string contains one or more actions of the form `attribute=value`, separated by `&`-signs:

`classify="testcase" : classify testcase`. The (URL-encoded) instance may be enclosed in single or double quotes. This action may be repeated to classify several instances in one request.

`set=option : set a TiMBL option for this request, as with the set command of the TCP protocol; a leading - may be omitted (e.g. set=k 3).`

`show=settings` or `show=weights` : include the current settings, or the current feature weights, in the XML answer.

For example, with the configfile of Section 5.1.2 running on `localhost:7000` with `protocol=http`:

```
> curl 'http://localhost:7000/dimin0?classify="=,=,=,=,=,=,=,+,p,e,=?"'
```

returns an XML document along the lines of

```
<?xml version="1.0"?>
<TiMblResult algorithm="IB1">
  <classification>
    <input>=,=,=,=,=,=,=,+,p,e,=?</input>
    <category>T</category>
    <distribution>{ T 1.00000 }</distribution>
    <distance>0</distance>
  </classification>
</TiMblResult>
```

where, as in the TCP protocol, the presence of the `distribution`, `distance`, `confidence` and `match_depth` elements depends on the verbosity settings of the base.

6.3 The JSON protocol

Since version 1.13, `TimblServer` can communicate in JSON. With `protocol=json`, client and server exchange newline-delimited JSON objects over the socket: every request is a single JSON object on one line, and every answer is a single JSON object (or array) on one line. **Note:** the JSON syntax might still change in future versions, so handle with care.

On connecting, the server sends a greeting. When the server hosts a single (default) base, the greeting is:

```
{"status":"ok"}
```

When several bases are available, the greeting lists them, and a base must be selected before classifying:

```
{"status":"ok","available_bases":["dimin0","dimin1","dimin2"]}
```

Every request object has a "command" key, and optionally a "param" key (a single string) or a "params" key (an array of strings). The commands mirror those of the TCP protocol:

base

select a base, e.g. {"command": "base", "param": "dimin0"}.
The server answers {"base": "dimin0"}, or an error object.

classify

classify one instance given as "param", or several instances given as "params":

```
{"command": "classify", "param": "=,=,=,=,=,=,+,p,e,=?"}
{"command": "classify", "params": ["-,=,=,=,+,k,e,=?", "=,=,=,=,+,p,e,=?"]}
```

The answer for a single instance is a JSON object; for multiple instances it is a JSON array of such objects. Each result object has a "category" key, and, depending on the verbosity settings of the base (+vdi, +vdb, +vmd, +vcf, +vn), the additional keys "distance", "distribution", "match_depth", "confidence" and "neighbors":

```
{"category": "T", "distance": 0.0, "distribution": "{ T 1.00000 }"}
```

set

set TiMBL options for this session, e.g. {"command": "set", "param": "-k 3"}. The server answers {"status": "ok"} or an error object.

query (or show)

with "param" "settings" or "weights", return the current settings or the current feature weights of the selected base as a JSON object, e.g. {"command": "query", "param": "weights"} returns

```
{"weighting": "gr", "weights": [0.024891, 0.021873, ..., 0.643681]}
```

exit

close the session. The server answers {"status": "closed"}.

Whenever a request cannot be handled, the server answers with an error object:

```
{"status": "error", "message": "Unknown command: 'clasify'"}
```

A complete session with the configfile of Section 5.1.2 (with protocol=json) looks like this, with lines sent by the client marked >:

```
{ "status": "ok", "available_bases": [ "dimin0", "dimin1", "dimin2" ] }
> { "command": "base", "param": "dimin0" }
{ "base": "dimin0" }
> { "command": "classify", "param": "=,=,=,=,=,=,+,p,e,=?" }
{ "category": "T", "distance": 0.0, "distribution": "{ T 1.00000 }" }
> { "command": "exit" }
{ "status": "closed" }
```

Chapter 7

The timblclient program

The distribution includes `timblclient`, a simple client program for the TCP protocol, which can be used to test a running server or to classify a test file against it:

```
> timblclient -h host -p port [-i inputfile] [-o outputfile] [--batch] [-b  
basename]
```

`-h <host>` : connect to the server on 'host'.

`-p <port>` : connect to 'port' on 'host'.

`-i <inputfile>` : a series of `timblserver` commands, as described in Section 6.1. Default is standard input.

`-o <outputfile>` : where to put the results. Default is standard output.

`--batch` : when `--batch` is specified, the input from 'inputfile' is not interpreted as `timblserver` commands, but as a 'normal' Timbl test file: every line is sent to the server 'as is', prepended by a `classify` instruction.

`-b <basename>` : select 'basename' on the server before testing.

For example, classifying a complete test file against the server from the Quick-start Tutorial (Chapter 4):

```
> timblclient -h localhost -p 7000 --batch -i dimin.test -o dimin.out
```


Bibliography

- [1] Walter Daelemans, Jakub Zavrel, Ko van der Sloot, and Antal van den Bosch (2026). *TiMBL: Tilburg Memory-Based Learner, version 7.0, Reference Guide*. Available from https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl_7.0_Manual.pdf.
- [2] Walter Daelemans and Antal van den Bosch (2005). *Memory-Based Language Processing*. Cambridge University Press, Cambridge, UK.